

Parcial 1 - Programación III

REVISAR QUÉ ARCHIVOS SE ENTREGAN. SI LA ENTREGA NO SE REALIZA DE MANERA CORRECTA, LA NOTA FINAL ES 1 (UNO).

-Consigna

Sistema de Gestión de Capturas Pokémon

Una liga Pokémon desea informatizar su sistema de registro de pokémones y capturas realizadas por entrenadores. La administración necesita poder:

- Registrar pokémones disponibles.
- Registrar capturas de pokémones.
- Liberar capturas realizadas.
- Consultar capturas por entrenador.
- Consultar pokémones por tipo.

1- CRUD de Pokémons (20 puntos)

Implementar alta, baja, modificación y consulta de pokémones.

Cada pokémon tiene:

- Nombre
- Tipo principal
- Tipo secundario
- Nivel base
- Cantidad disponible

Ejemplos de tipos: Fuego, Agua, Planta, Eléctrico, Roca, Tierra

2- Registrar una nueva captura (30 puntos)

Implementar la posibilidad de registrar una captura de un pokémon por parte de un entrenador.

Cada captura debe registrar:

- Entrenador
- Pokémon
- Fecha de captura
- Nivel del pokémon capturado
- Estado de la captura (estado inicial de una captura exitosa será: ACTIVA)

Validaciones obligatorias:

- El entrenador debe existir.
- El pokémon debe existir.
- Debe haber al menos 1 unidad disponible del pokémon.
- La fecha de captura no puede ser posterior a la fecha actual.

Si alguna validación falla, se debe lanzar una excepción propia llamada: **CapturaInvalidaException**
Si la captura es exitosa:

- El estado será ACTIVA.
- Se debe descontar 1 unidad disponible del pokémon.

3- Liberar una captura (20 puntos)

Implementar la posibilidad de liberar un pokémon capturado.
Para eso, dado el ID de una captura:

- La captura debe existir.
- La captura debe estar en estado ACTIVA.
- El estado debe cambiar a LIBERADA.
- Se debe incrementar nuevamente en 1 la cantidad disponible del pokémon.
- Si la captura no existe o no está activa, se debe lanzar: **CapturaInvalidaException**

4- Listar capturas de un entrenador (15 puntos)

Dado el ID de un entrenador, mostrar todas sus capturas.
La respuesta debe mostrar como mínimo:

- Nombre del entrenador
- Ciudad del entrenador
- Nombre del pokémon capturado
- Tipo principal del pokémon
- Fecha de captura
- Nivel capturado
- Estado de la captura

5- Consultar pokémones por tipo (15 puntos)

Dado un tipo de pokémon, por ejemplo "Fuego" o "Agua", mostrar todos los pokémones registrados que tengan ese tipo como tipo principal o tipo secundario.
Este punto debe resolverse obligatoriamente utilizando: Streams

- Base de datos

```
CREATE DATABASE liga_pokemon;
```

```
USE liga_pokemon;
```

```
CREATE TABLE entrenador (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nombre_completo VARCHAR(100),  
    ciudad VARCHAR(100),  
    cantidad_medallas INT,  
    gimnasio_favorito VARCHAR(100)  
);
```

```
CREATE TABLE pokemon (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nombre VARCHAR(100),  
    tipo_principal VARCHAR(50),  
    tipo_secundario VARCHAR(50),  
    nivel_base INT,  
    cantidad_disponible INT  
);
```

```
CREATE TABLE captura (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    id_entrenador INT,  
    id_pokemon INT,  
    fecha_captura DATE,  
    nivel_capturado INT,  
    estado ENUM('ACTIVA', 'LIBERADA', 'CANCELADA'),  
  
    FOREIGN KEY (id_entrenador) REFERENCES entrenador(id),  
    FOREIGN KEY (id_pokemon) REFERENCES pokemon(id)  
);
```

- Datos de prueba sugeridos

```
INSERT INTO entrenador  
(nombre_completo, ciudad, cantidad_medallas, gimnasio_favorito)  
VALUES  
('Ash Ketchum', 'Pueblo Paleta', 8, 'Gimnasio de Ciudad Plateada'),  
('Misty Waterflower', 'Ciudad Celeste', 4, 'Gimnasio de Ciudad Celeste'),  
('Brock Slate', 'Ciudad Plateada', 5, 'Gimnasio de Ciudad Plateada');
```

```
INSERT INTO pokemon  
(nombre, tipo_principal, tipo_secundario, nivel_base, cantidad_disponible)  
VALUES  
('Pikachu', 'Eléctrico', NULL, 5, 3),
```

```
('Charmander', 'Fuego', NULL, 5, 2),  
( 'Bulbasaur', 'Planta', 'Veneno', 5, 2),  
( 'Squirtle', 'Agua', NULL, 5, 2),  
( 'Geodude', 'Roca', 'Tierra', 8, 4);
```

-Aclaraciones

- El proyecto debe compilar correctamente.
- No se pide CRUD de entrenadores. Los entrenadores pueden cargarse directamente desde la base de datos con los inserts provistos.
- Los entrenadores pueden cargarse previamente en la base de datos.
- No alcanza con crear las entidades: el sistema debe poder probarse desde Postman o herramienta similar.
- Se permite el uso de Lombok.
- Se valorará la claridad del código, la separación en capas y el correcto uso de JPA.